

Styling with VueJS & Functions

Dr. B. Prabavathy
Associate Professor



Styling with VueJS

In Vue.js, **styling means applying CSS to Vue components**. Vue provides several ways to style elements dynamically.

The important approaches are:

- Normal CSS
- `<style>` inside a component
- Dynamic classes using `:class`
- Dynamic styles using `:style`
- Conditional CSS classes

<code><style></code>	Write CSS inside the Vue component
<code>:class</code>	Dynamically apply CSS classes
<code>:style</code>	Dynamically apply CSS properties
Conditional <code>:class</code>	Change styling based on a condition

1. Normal CSS

You can write CSS normally in a .css file and import it into Vue.

```
<h1 class="title">Student Details</h1> in  
App Component  
<p class="para">Department: {{  
department }}</p> in Student Component
```

Style in main.css - global style sheet
applied for any HTML element in any
component in Vue Application

main.css

```
.title {  
  color: blue;  
  font-size: 30px;  
}  
.para{  
  color: red;  
  font-size: 30px;  
}
```

Student Details

Anitha

Department: Computer
Science

 Student Photo

Status: Active

Courses

- Vue.js
- Java
- Python

Completed Courses: 2

Complete Course

2. <style> in a Vue component

CSS can also be written directly inside a .vue component.

```
<template>  
  <h1>Welcome to Vue.js</h1>  
</template>
```

```
<style scoped>  
h1 {  
  color: green;  
  font-size: 30px;  
}  
</style>
```

The <style> section contains the CSS for the component.

3. Dynamic classes using :class

Vue allows the class to change based on a data value.

```
<script setup>
import { ref } from 'vue'
const isActive = ref(true)
</script>
<template>
  <p :class="{ active: isActive }">
    Welcome to Vue.js
  </p>
</template>
<style>
p{
  font-size:30px;
}
.active {
  color: green;
  font-weight: bold;
}
</style>
```

Here:

```
:class="{ active: isActive }"
```

means:

5 Apply the active CSS class **when** ^{v12}isActive is true.

```
const isActive = ref(true)
```

Student Details

Welcome to Vue.js

```
const isActive = ref(false)
```

Student Details

Welcome to Vue.js



4. Dynamic styles using :style

Vue can directly change CSS properties using data.

```
<script setup>
import { ref } from 'vue'
const textColor = ref('red') →
</script>
<template>
  <p :style="{ color: textColor }">
    This text is red
  </p>
</template>
```

Student Details

Welcome to Vue.js

This text is red

If textColor changes to "blue", the text automatically becomes blue.

```
const textColor = ref('blue') →
```

Student Details

Welcome to Vue.js

This text is red



5. Conditional classes

A common scenario is changing the appearance based on a condition

```
<script setup>
import { ref } from 'vue'
const isOnline = ref(true) ➔
</script>
<template>
  <p :class="isOnline ? 'online' : 'offline'">
    User Status
  </p>
</template>
<style>
.online {
  color: green;
}
.offline {
  color: red;
}
</style>
```

If:

isOnline = true

the online class is applied.

If:

isOnline = false

the offline class is applied.

Student Details

Welcome to Vue.js

This text is red

User Status

const isOnline = ref(false) ➔

Student Details

Welcome to Vue.js

This text is red

User Status

Exercise:

Develop a Student Attendance Dashboard using Vue.js. The dashboard should display each student's name, department, and attendance percentage.

Implement the following styling requirements:

1. Use normal CSS to style the overall page and student details.
2. Use the `<style>` section inside the Vue component to define the component's CSS.
3. Use `:class` to dynamically apply:
 - present class when attendance is 75% or above.
 - low-attendance class when attendance is below 75%.
4. Use `:style` to dynamically set the font size of the student's name.
5. Use a conditional class to display the student's attendance status in different styles:
 - Green → "Eligible"
 - Red → "Not Eligible"



In Vue.js, functions are used to perform an action when something happens, such as clicking a button, entering data, or changing a value.

Example 1: Simple function

```
<script setup>  
function sayHello() {  
  alert("Hello, Welcome to Vue.js!")  
}  
</script>
```

```
<template>  
  <button @click="sayHello">  
    Click Me  
  </button>  
</template>
```

Here:

- `function sayHello()` → defines a function.
- `@click="sayHello"` → calls the function when the button is clicked.
- `alert()` → displays the message.

Example 2: Function with a parameter

```
<script setup>  
function greet(name) {  
  alert("Hello " + name)  
}  
</script>
```

```
<template>  
  <button @click="greet('Praba')">  
    Greet  
  </button>  
</template>
```

When the button is clicked:

Hello Praba

Here, 'Praba' is passed as an argument to the function.

Example 3: Function that changes a value

```
<script setup>
import { ref } from 'vue'
const count = ref(0)
function increment() {
  count.value++
}
</script>
<template>
  <p>Count: {{ count }}</p>
  <button @click="increment">
    Increase
  </button>
</template>
```

Every time the button is clicked:
0 → 1 → 2 → 3 → ...

Example 4: Function for student attendance

```
<script setup>
import { ref } from 'vue'
const attendance = ref(70)
function increaseAttendance() {
  attendance.value += 5
}
</script>
<template>
  <h2>Student Attendance</h2>
  <p>Attendance: {{ attendance }}%</p>
  <button @click="increaseAttendance">
    Mark Attendance
  </button>
</template>
```

If the button is clicked:

12 70% → 75% → 80% → 85%_{v1.2}

